

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Модули для работы с файловой системой: OS и SHUTIL

Модуль `os` — это библиотека функций для работы с операционной системой.

Полную документацию по модулю можете изучить на официальном [сайте](#).

Получение информации об ОС

Импортируем модуль `os`, и применяем метод `name`.

```
import os
print(os.name)
```

```
1 x
posix
```

Получили на выходе, наверное, совсем не то что ожидали. POSIX - это стандарт интерфейса, подробнее в [wikipedia](#), а мы хотели увидеть название ОС.

Для определения типа ОС можно воспользоваться встроенным модулем `sys` и методом `platform`.

```
import sys
print(sys.platform)
```

```
1 x
linux
```

Информация слишком общая, если мы хотим узнать больше подробностей, то ещё одним решением будет использование модуля `platform` и функции `release()`.

```
import platform
print(platform.system())
print(platform.release())
```

```
1 x
Linux
4.19.0-kali5-amd64
```

На самом деле через модуль `os` одним запросом мы можем получить сразу же много информации о системе. Для этого всего лишь нужно изменить команду на `uname`.

Метод `uname()` — выводит информацию о ОС, возвращает объект с атрибутами: `sysname` - имя операционной системы, `nodename` - имя машины в сети, `release` - релиз, `version` - версия, `machine` - идентификатор машины.

```
import os
print(os.uname())
```

1 x

```
posix.uname_result(sysname='Linux', nodename='exp', release='4.19.0-kali5-
```

Или выведем данные в столбик.

```
import os
print('\n'.join(os.uname()))
```

1 x

```
Linux
exp
4.19.0-kali5-amd64
#1 SMP Debian 4.19.37-5kali1 (2019-06-20)
x86_64
```

Получение текущей директории

Метод `getcwd()` возвращает текущую директорию, то есть ту в которой мы находимся.

```
import os

print("Текущая директория:", os.getcwd())
```

my_os x

```
Текущая директория: /root/PycharmProjects/education/pro_2
```

Аналогичный результат мы можем получить так:

```
import os

print("Текущая директория:", os.environ['PWD'])
```

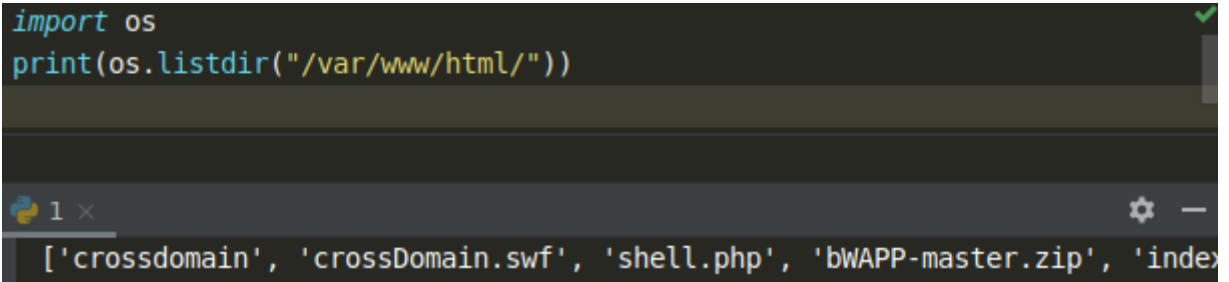
my_os x

```
Текущая директория: /root/PycharmProjects/education/pro_2
```

Получение содержимого директорий

Метод `listdir()` возвращает список в произвольном порядке, содержащий имена записей в каталоге (каталогов и файлов), с заданных путем.

```
import os
print(os.listdir("/var/www/html/"))
```

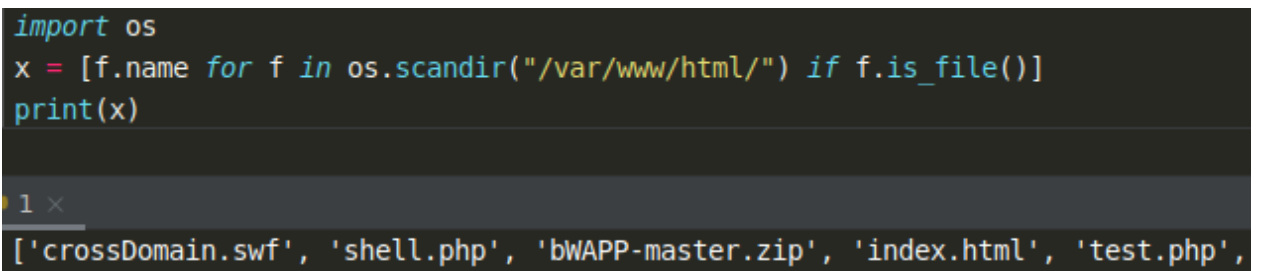


Метод `scandir()` возвращает итератор `os.DirEntry` и выводит список в произвольном порядке, содержащий имена записей в каталоге, с заданных путем. С помощью различных атрибутов для `DirEntry` мы можем получать различные результаты.

Метод `is_file()` - возвращает `True`, если запись является файлом или символической ссылкой, указывающей на файл; возвращает `False` если запись это каталог или другая нефайловую запись, или если она больше не существует.

Выведем список файлов.

```
import os
x = [f.name for f in os.scandir("/var/www/html/") if f.is_file()]
print(x)
```



Метод поддерживает использование менеджера контекста `with`. Тот же пример с применением менеджера контекста.

```
import os
with os.scandir("/var/www/html/") as i:
    for f in i:
        if f.is_file():
            print(f.name)
```

```
1 x
123.php
index.nginx-debian.html
product.php
index.php
crossDomain.as
flash_1582052976_1011556249.log
routed.php
flash_1582053822_489203359.log
crossdomain.xml
```

Метод `is_dir()` - возвращает `True` если эта запись является каталогом или символической ссылкой, указывающей на каталог; возвращает `False` если запись это файл, или если он больше не существует.

Выведем список директорий.

```
import os
x = [f.name for f in os.scandir("/var/www/html/") if f.is_dir()]
print(x)
```

```
1 x
['crossdomain', 'images', 'modx', 'bWAPP-master']
```

А с помощью метода `walk()` можно вывести содержимое любого каталога, включая файлы и каталоги которые содержатся в нём. Этот метод принимает 1 аргумент – имя каталога.


```
import os

for dirpath, dirnames, filenames in os.walk("/var/www/html"):
    # перебрать каталоги
    for i in dirnames:
        print("Каталог:", os.path.join(dirpath, i))
    # перебрать файлы
    for i in filenames:
        print("Файл:", os.path.join(dirpath, i))
```

```
proverka x
Каталог: /var/www/html/crossdomain
Каталог: /var/www/html/images
Каталог: /var/www/html/modx
Каталог: /var/www/html/bWAPP-master
Файл: /var/www/html/crossDomain.swf
Файл: /var/www/html/shell.php
Файл: /var/www/html/data.php
Файл: /var/www/html/bWAPP-master.zip
```

Создание директорий

Для создания директории применяется метод `mkdir()`.

```
import os
os.mkdir("/var/www/html/test")
```

Методом `makedirs()` можно создавать сразу несколько новых папок, если предыдущая директория является родительской для следующей.

```
import os
os.makedirs("/var/www/html/test/1/2/3")
```

Заглядываем в директорию и видим, что всё прошло успешно.

```
Вид  Переход  Справка
🏠  /var/www/html/test/1/2/3/
```

Удаление файлов и директорий

Метод `rmdir()` удалит пустой каталог.

```
import os
os.rmdir("/var/www/html/test/1/2/3")
```

Для удаления множества пустых папок следует вызывать функцию `removedirs()`. Она предоставляет возможность избавиться сразу от нескольких каталогов на диске, при условии, что все они вложены друг в друга. Таким образом, указав путь к конечной папке, можно легко удалить все родительские директории, но только если они будут пустыми.

```
import os
os.removedirs("/var/www/html/test/1/2")
```

Удалить ненужный файл можно методом `remove()`.

```
import os
os.remove("/var/www/html/flash_1582053822_489203359.log")
```

Переименование файла или директории

Метод `rename()` предназначен для переименования файла или директории.

```
import os
os.rename("/var/www/html/test.php", "/var/www/html/test2.php")
os.rename("/var/www/html/images", "/var/www/html/images2")
```

Проверка прав доступа к файлу или папке

Проверка прав для всех пользователей и групп.

```
import os
import stat

print(oct(stat.S_IMODE(os.lstat("/var/www/html/bWAPP-master").st_mode)))
```

```
1 x
0o777
```

Задать права можно через метод `chmod()` добавив префикс `0o`, поскольку разрешения задаются как восьмеричное целое число, а Python автоматически обрабатывает любое целое число с начальным нулем как восьмеричное.

```
import os
os.chmod("/var/www/html/test.php", 0o777)
```

Подробную информацию по правам пользователя можно посмотреть [ЗДЕСЬ](#)

Выполнение команд

Метод `system()` - исполняет системную команду, возвращает код её завершения (в случае успеха 0).

Это очень мощный метод, позволяющий вводить любые команды, и работать как если бы вы это делали прямо в терминале.

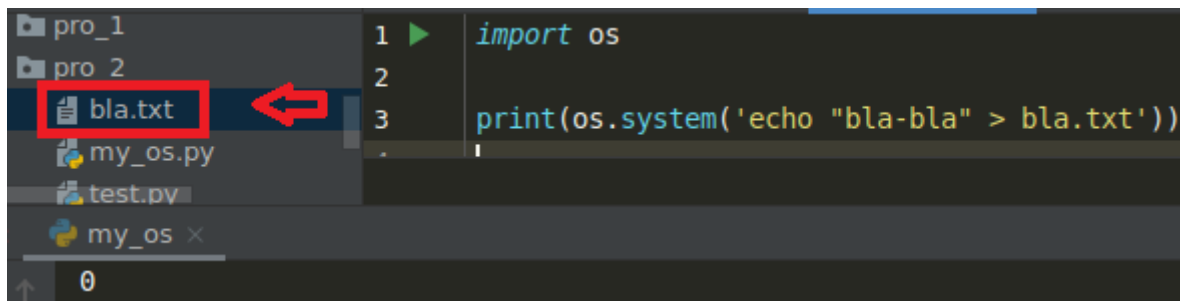
```
import os
print(os.system("pwd"))
print(os.system("id"))
```

1 x

```
/root/PycharmProjects/education
0
uid=0(root) gid=0(root) группы=0(root),142(vboxsf),999(docker),65534(nogroup)
0
```

Кроме выполнения системных команд, мы можем делать как уже было написано всё что позволяет делать терминал. Например, создать новый файл уже содержащий запись.

Вданном примере получится файл bla.txt со строкой bla-bla.



```
1 import os
2
3 print(os.system('echo "bla-bla" > bla.txt'))
```

Создадим файл `test.py` с одной строкой кода `print('Проверка!')` В этой же директории создадим другой файл со следующим содержимым и запустим:

```
import os

program = "python"
arguments = ["test.py"]
print(os.execvp(program, (program,) + tuple(arguments)))
```

my_os x

```
Проверка!

Process finished with exit code 0
```

Получили результат выполнения программы `test.py` подобно тому, как если бы мы импортировали модуль `test.py`. Метод `execvp()` принимает в качестве аргумента другую программу и выполняет её.

Мы можем запустить и программу, находящуюся в другом каталоге, просто указав полный путь.


```
import os

program = "python"
arguments = ["/root/PycharmProjects/education/pro_1/proverka.py"]
print(os.execvp(program, (program,) + tuple(arguments)))
```

my_os ×

Проверка!

Process finished with exit code 0

Класс `os.path`. Работа с путями, папками и файлами

Класс `os.path()` является вложенным модулем в модуль `os`, и реализует разные функции для работы с путями.

С помощью метода `exists()` который возвращает `True` или `False`, мы можем проверить наличие файла или директории, чтобы избежать ошибок.

```
import os
print(os.path.exists("/var/www/html/flash_1582053822_489203359.log"))
print(os.path.exists("/var/www/html/login.php"))
```

1 ×

False

True

Ещё два удобных метода `os.path.isfile` и `os.path.isdir` – первый позволяет проверить, является ли переданный ему объект файлом, второй проверяет, является ли он папкой. Используйте их в условиях `if` для проверки существования объектов.

Так же класс `os.path` позволяет создавать пути к файлам, не заботясь, о той системе, в который вы находитесь.

Напоминаю, что для отделения папок и файлов в путях в **Windows** используется обратный слэш - \ В ***nix** системах используется прямой слэш - / или просто слэш.

Для того чтобы не беспокоиться, об этом символе при создании пути, был добавлен метод `join`

Запуск кода в Windows

```
1 import os
2
3 target_file = "my_file.txt"
4 full_path = os.path.join(os.getcwd(), target_file)
5 print(f'{full_path=}')

Run: os_shutil x
full_path='C:\\tmp\\test_proj\\my_module\\my_file.txt'
```

Запуск кода в Linux

```
>>> import os
...
... target_file = "my_file.txt"
... full_path = os.path.join(os.getcwd(), target_file)
... print(f'{full_path=}')
full_path='/tmp/my_file.txt'
```

Метод `join` сам выберет нужный символ для отделения и вам не придётся задумываться о системе, на которой вы запускаете ваш скрипт.

Ещё один очень удобный метод в классе `path` – `split`. Он возвращает 2 объекта: папку файла и имя файла (если переданный аргумент оканчивается слэшем, второй объект будет пустой строкой):

```
>>> import os
...
... target = '/usr/bin/nano'
... empty_target = '/usr/bin/'
...
... print(f'{os.path.split(target)=}')
... print(f'{os.path.split(empty_target)=}')
os.path.split(target)=('/usr/bin', 'nano')
os.path.split(empty_target)=('/usr/bin', '')
```

Второй изучаемый модуль - **shutil** содержит дополнительные функции и абстракции для работы с файлами и директориями, которых нет в модуле **os**.

Полная документация, как всегда находится на официальном [сайте](#).

Из ряда преимуществ модуля **shutil** по сравнению с модулем **os** можно отметить:

- **Более удобные функции копирования и перемещения:** Модуль **shutil** содержит функции **copy()** и **move()**, которые позволяют копировать и перемещать файлы и директории с одной строкой кода. В модуле **os** для выполнения аналогичных операций требуется больше кода и проверок.

```
>>> import shutil
2 import os
3
4 target_file = '/usr/bin/cat'
5 output_path = '/tmp/python/'
6
7 # указываем 2 аргумента: путь к файлу и путь места назначения
8 shutil.copy(target_file, output_path)
9
10 print(f'Отображаем содержимое папки {output_path}:')
11 os.system(f'ls {output_path}')
Отображаем содержимое папки /tmp/python/:
cat
```

- **Рекурсивное копирование и удаление:** Модуль **shutil** содержит функции **copytree()** и **rmtree()**, которые автоматически копируют или удаляют все содержимое директории, включая вложенные файлы и поддиректории. В модуле **os** необходимо использовать рекурсивные вызовы и манипуляции с путями, чтобы достичь аналогичного результата.
- **Атомарные операции перемещения:** Функция **move()** в модуле **shutil** содержит атомарную операцию перемещения файлов и директорий. Это означает, что операция перемещения будет либо выполнена полностью, либо не будет выполнена вообще. В случае с модулем **os**, для создания атомарных операций перемещения требуется дополнительный код.

- **Создание архивов:** Модуль `shutil` содержит функцию `make_archive()`, которая позволяет создавать архивы с различными форматами (например, ZIP, TAR). В модуле `os` требуется больше кода и библиотек для создания архивов.

```
>>> import shutil
...
... main_path = '/tmp/python/'
... output_path = os.path.join(main_path, 'my_archive')
...
... # Указываем путь к файлу архива, тип архива и
... # ту папку, содержимое которой нужно упаковать
... shutil.make_archive(output_path, 'bztar', '/etc/apt/')
...
... print(*os.listdir(main_path), sep='\n')
my_archive.tar.bz2
cat
```

- **Удобная работа с путями:** Модуль `shutil` содержит функции, которые обрабатывают пути к файлам и директориям автоматически. Например, функция `copy()` позволяет указывать только имена файлов и автоматически определит путь назначения. В модуле `os` требуется больше кода для работы с путями.

Хотя модуль `shutil` содержит удобные функции для работы с файлами и директориями, модуль `os` все равно остается полезным для выполнения других операций на более низком уровне или для работы с операционной системой напрямую.